Cetus

Audit Report

contact@bitslab.xyz

https://twitter.com/movebit_

Thu Jan 22 2026

Cetus Audit Report

1 Executive Summary

1.1 Project Information

Description	Cetus is an aggregator contract that interacts with Suilend.
Type	DeFi
Auditors	Sprig,Leon@BitsLab
Timeline	Sun Jan 04 2026 - Wed Jan 07 2026
Languages	Move
Platform	Sui
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/CetusProtocol/cetus-margin-trading
Commits	b50b5eef220434a675e0cda862ec49d3880fb28d c8b639dbb7c7818bda4db65fc938f9fd0bff40f8 612f8f38b959f132128666e2c27bd656b82fe5b2 ddf6408badbaefe9da58822108137f343b73e2f0 3e2540638972bc81f8bf3fbc7c1e7b3a83543a58 d8b6e13ac981eeab038aa6f4faf37c0ca48a2a76 119a46b4b66887c1a32deb33c2c4e24b3d8bc42f

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
ACA	sources/admin_cap.move	8fc585a3b1ee2541644c08ff403edc4172d0aacd
CON	sources/context.move	6600ba5fc9d577bf13382da09e4b7fcb4a65b572
ERR	sources/errors.move	8cb2477e4e3d2f7cdb1d17299d1262152ef4fc3d
POS	sources/position.move	c1f8a59e80cf1a367933ad4ed55533357cdab721
VER	sources/versioned.move	14ab956a9bd175d980c90dfc6d9fefb7edafb1a
ROU	sources/router.move	04c60b73f9525e60210cfb9f454df83a82b75245
CON1	sources/config.move	4c97a965ccc63f152537a4267e4f3f9471e9cc36
MAR	sources/market.move	2fafa15923ef62169068c060b7971ff2cb2f19a3
PER	sources/permissions.move	b9c1259a1c9fe2ba3fb70c1d67ee74a5609b8d44

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	8	8	0
Critical	1	1	0
Major	1	1	0
Medium	1	1	0
Minor	5	5	0
Informational	0	0	0

1.4 MoveBit Audit Breakdown

MoveBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values
- The flow of capability
- Witness Type

1.5 Methodology

The security team adopted the **"Testing and Automated Analysis"**, **"Code Review"** and **"Formal Verification"** strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Formal Verification(Optional)

Perform formal verification for key functions with the Move Prover.

(4) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by Cetus to identify any potential issues and vulnerabilities in the source code of the Cetus smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 8 issues of varying severity, listed below.

ID	Title	Severity	Status
CON-1	Incorrect Permission Verification	Critical	Fixed
ERR-2	Duplicate Error Code Definitions	Minor	Fixed
MAR-4	Issues in Market Key Generation	Major	Fixed
ROU-6	Missing Non-Zero Validation in Borrow Function	Minor	Fixed
ROU-7	Event Spam in Margin Trading Functions	Minor	Fixed
ROU-8	Event Trace Manipulation	Medium	Fixed
ROU-9	Missing Version Check in `create_margin_trading_context`	Minor	Fixed
ROU-10	Missing Consistency Check in `close_position`	Minor	Fixed

3 Participant Process

Here are the relevant actors with their respective abilities within the Cetus Smart Contract :
Admin

- Admin can set the collect obligation owner cap address through the `set_collect_obligation_owner_cap_address()` function.
- Admin can update global open position permissions through the `update_global_open_position_permissions()` function.
- Admin can update global close position permissions through the `update_global_close_position_permissions()` function.
- Admin can update global deposit permissions through the `update_global_deposit_permissions()` function.
- Admin can update global borrow permissions through the `update_global_borrow_permissions()` function.
- Admin can update global withdrawal permissions through the `update_global_withdraw_permissions()` function.
- Admin can update global repay permissions through the `update_global_repay_permissions()` function.
- Admin can create a new margin trading context through the `new_margin_trading_context()` function.
- Admin can create a market through the `create_market()` function.
- Admin can update market open position permission through the `update_market_open_position_permission()` function.
- Admin can update market close position permission through the `update_market_close_position_permission()` function.
- Admin can update market deposit permission through the `update_market_deposit_permission()` function.

- Admin can update market borrow permission through the `update_market_borrow_permission()` function.
- Admin can update market withdrawal permission through the `update_market_withdraw_permission()` function.
- Admin can update market repay permission through the `update_market_repay_permission()` function.
- Admin can set fee rates through the `set_fee_rate()` function.
- Admin can claim accumulated fees through the `claim_fees()` function.

User

- User can open positions through the `open_position()` function.
- User can close positions through the `close_position()` function.
- User can borrow assets through the `borrow()` function.
- User can repay loans through the `repay()` function.
- User can withdraw collateral through the `withdraw()` function.
- User can deposit collateral through the `deposit()` function.
- User can confirm trading actions through the `confirm()` function.

4 Findings

CON-1 Incorrect Permission Verification

Severity: Critical

Status: Fixed

Code Location:

sources/config.move#261-283

Descriptions:

Functions like `check_xxx_permission()` are used to check if the `GlobalConfig` has the correct permission. However, the call to `check_xxx_permission()` does not use an assertion to verify the return value. All permission-checking functions in `permissions` return a `bool` rather than aborting when permissions are absent. The missing verification of the return value here renders all checks in `config` ineffective.

sources/config.move

```
//@Audit Incorrect: No assertion used here
public fun check_open_permission(config: &GlobalConfig): bool {
    permissions::check_permission(config.permissions,
permissions::open_position_permission())
}

public fun check_close_permission(config: &GlobalConfig): bool {
    permissions::check_permission(config.permissions,
permissions::close_position_permission())
}

public fun check_deposit_permission(config: &GlobalConfig): bool {
    permissions::check_permission(config.permissions,
permissions::deposit_permission())
}

public fun check_borrow_permission(config: &GlobalConfig): bool {
```

```
    permissions::check_permission(config.permissions,
permissions::borrow_permission())
}

public fun check_withdraw_permission(config: &GlobalConfig): bool {
    permissions::check_permission(config.permissions,
permissions::withdraw_permission())
}

public fun check_repay_permission(config: &GlobalConfig): bool {
    permissions::check_permission(config.permissions, permissions::repay_permission())
}
```

Suggestion:

It is recommended to use assert to complete the missing assertions.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

ERR-2 Duplicate Error Code Definitions

Severity: Minor

Status: Fixed

Code Location:

sources/errors.move#111-113

Descriptions:

The functions `err_invalid_leverage_too_high` and `err_leverage_too_high` both abort with the same error code `EInvalidLeverageTooHigh`, which may lead to unclear error handling and confusion.

Suggestion:

It is recommended to consolidate duplicate error functions or assign distinct error codes for different scenarios.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

MAR-4 Issues in Market Key Generation

Severity: Major

Status: Fixed

Code Location:

sources/market.move#146-192

Descriptions:

`new_market_key` is used to generate a unique ID for each base quote token pair. The function contains multiple critical vulnerabilities and logic errors:

1. Array Index Out-of-Bounds Risk: When coin type names share a common prefix (e.g., USD/USDC or ETH/ETH_Wormhole), the while loop may continue incrementing `i` beyond the length of the shorter name vector, leading to out-of-bounds access when calling `vector::borrow`.
2. Incorrect Index Increment Order: The variable `i` is incremented before accessing the vectors, potentially skipping the first character comparison and disrupting the intended byte-by-byte comparison logic.
3. Incorrect Length Parameter in Second Loop: The second while loop uses `len_quote` as the loop bound but attempts to access `first` with index `i`, which could exceed `len_base` when `len_base < len_quote`.
4. Coin Pair Creation Failure for Prefix Relationships: The algorithm fundamentally fails to generate valid market keys when one coin name is a prefix of another (e.g., "USD" and "USDC"), as the prefix comparison logic doesn't handle this case properly.

sources/market.move

```
...

public(package) fun new_market_key(base: TypeName, quote: TypeName): ID {
    let coin_type_base = *ascii::as_bytes(&type_name::into_string(base));
    let coin_type_quote = *ascii::as_bytes(&type_name::into_string(quote));
    let (len_base, len_quote) = (vector::length(&coin_type_base),
```

```
vector::length(&coin_type_quote));

    let mut i = 0;
    let mut first_base_byte = *vector::borrow(&coin_type_base, i);
    let mut first_quote_byte = *vector::borrow(&coin_type_quote, i);

    while (first_base_byte == first_quote_byte) { //@Audit Array Index Out-of-Bounds Risk
        i = i + 1;//@Audit Incorrect Index Increment Order
        first_base_byte = *vector::borrow(&coin_type_base, i);
        first_quote_byte = *vector::borrow(&coin_type_quote, i);
    };

    let (mut first, second) = if (first_base_byte < first_quote_byte) {
        (coin_type_base, coin_type_quote)
    } else {
        (coin_type_quote, coin_type_base)
    };

    let mut i = 0;
    let mut check_pass = false;

    // Check BASE must come before QUOTE (lexicographic order)
    while (i < len_quote) { //@Audit Incorrect Length Parameter
        let byte_quote = *vector::borrow(&second, i);
        if (!check_pass && i < len_base) {
            let byte_base = *vector::borrow(&first, i);
            if (byte_base > byte_quote) {
                abort EInvalidCoinTypeSequence
            };
            if (byte_base < byte_quote) {
                check_pass = true
            };
        };
        vector::push_back(&mut first, byte_quote);
        i = i + 1;
    };

    if (!check_pass) {
        assert!(len_base != len_quote, ESameCoinType);
```

```
        assert!(len_base < len_quote, EInvalidCoinTypeSequence);
    };

    object::id_from_bytes(hash::blake2b256(&first))
}
```

Suggestion:

It is recommended to refactor the `new_market_key` function to ensure its functional correctness.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

ROU-6 Missing Non-Zero Validation in Borrow Function

Severity: Minor

Status: Fixed

Code Location:

sources/router.move#142-154

Descriptions:

The `borrow` function within the margin trading protocol is responsible for processing user borrow requests against their collateralized positions.

sources/router.move

```
public fun borrow<P, T>(
    context: &MarginTradingContext,
    config: &GlobalConfig,
    lending_market: &mut LendingMarket<P>,
    market: &Market,
    position_cap: &PositionCap,
    reserve_array_index: u64,
    amount: u64, //@Audit non-zero
    system_state: &mut SuiSystemState,
    clock: &Clock,
    versioned: &Versioned,
    ctx: &mut TxContext,
): Coin<T> {
```

This function accepts an `amount` parameter specifying the quantity of assets the user wishes to borrow. The code lacks a validation step to ensure the requested `amount` is greater than zero before proceeding with the borrow operation.

Suggestion:

It is recommended to add a check at the beginning of the function to revert the transaction if the provided `amount` is zero.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

ROU-7 Event Spam in Margin Trading Functions

Severity: Minor

Status: Fixed

Code Location:

sources/router.move#403-448

Descriptions:

The `create_margin_trading_context` function is designed to validate and create a context for margin trading actions, which is then finalized by the `confirm` function that emits an event and destroys the context. The `confirm` function unconditionally emits a `MarginTradingConfirmEvent` for any valid context it receives. Since the `create_margin_trading_context` function can be called by users to generate a valid context for any approved action string, and the resulting context can be passed to `confirm`, a user can repeatedly call this function pair.

This mechanism allows a user to generate and confirm an unbounded number of margin trading contexts without executing any actual state-changing trade logic.

This design flaw enables users to spam the network by emitting a high volume of `MarginTradingConfirmEvent` events at minimal cost.

Suggestion:

It is recommended to tightly couple the emission of the `MarginTradingConfirmEvent` with the actual execution of the corresponding margin trading operation that changes the protocol's state.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

ROU-8 Event Trace Manipulation

Severity: Medium

Status: Fixed

Code Location:

sources/router.move#404-443

Descriptions:

The `create_margin_trading_context` function is responsible for creating a context for margin trading operations. This context includes a `trace_id` parameter intended to be a unique identifier for correlating all events emitted throughout a specific trading workflow for off-chain monitoring and tracking purposes.

sources/router.move

```
// ===== Confirm Functions =====
public fun create_margin_trading_context<COIN_TYPE>(
    trace_id: String,//@Audit trace_id
    market: &Market,
    position_cap: &PositionCap,
    action: String,
    amount: u64,
    ctx: &mut TxContext,
): MarginTradingContext {
```

The function accepts this `trace_id` as a direct user input parameter without any validation or enforcement of uniqueness. This design allows a user to manually specify any string as the trace identifier. Consequently, a malicious user can intentionally reuse the same `trace_id` across unrelated or fraudulent transactions.

This action generates misleading on-chain events that appear to be part of a single, coherent workflow, effectively poisoning the event log. This undermines the integrity and reliability of the off-chain tracking and monitoring systems that depend on the uniqueness

and authenticity of the `trace_id` to accurately reconstruct and audit transaction sequences and user activities.

Suggestion:

It is recommended to implement a mechanism to generate a unique, non-guessable `trace_id` on-chain for each trading context, rather than relying on user-provided input.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

ROU-9 Missing Version Check in `create_margin_trading_context`

Severity: Minor

Status: Fixed

Code Location:

sources/router.move#446-483

Descriptions:

The `create_margin_trading_context` function is used to create a context for margin trading operations. It validates the provided action string against a list of allowed actions. However, the function does not check the version of the market or position cap objects. If the protocol upgrades and the data structure of these objects change, this function might create a context based on outdated or incompatible data. This could lead to incorrect state transitions during trading operations, potentially causing financial losses for users or protocol instability.

Suggestion:

It is recommended to add version checks for the market and position cap objects within the function to ensure compatibility with the current protocol version.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

ROU-10 Missing Consistency Check in `close_position`

Severity: Minor

Status: Fixed

Code Location:

sources/router.move#104-139

Descriptions:

The `close_position` function is used to close a trading position in the margin trading system. It removes the position from the market and performs checks on the associated lending obligation. However, the function does not verify the consistency between the provided `Position`, `Market`, and `LendingMarket` objects. Specifically, it does not confirm that the position being closed actually belongs to the correct market and lending market context passed into the function. This lack of validation means the function might incorrectly process a position from a different market or lending pool.

Suggestion:

It is recommended to add validation steps to ensure the position, market, and lending market references are consistent and belong to the same trading context before proceeding with the close operation.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.

